AI Summer Marketing Project Wiki

2025-06-19

Contents

Marketin Agent **3**
- Project Structure . . . 3
- Modules Overview . . . 4
  - modules/ . . . 4
  - core/ . . . 4
  - shared/ . . . 4
  - assets/ . . . 5
- Development Guidelines . . . 5
  - Code Organization . . . 5
  - State Management . . . 5
  - Naming Conventions . . . 5
- Redux Workflow . . . 5
- Best Practices . . . 5

Auth Module **6**
- Structure . . . 6
- Features . . . 6
- Usage . . . 6
  1. AuthProvider . . . 6
  2. Protecting Routes . . . 7
  3. Accessing Auth State and Actions . . . 7
  4. Login Example . . . 7
  5. API Service . . . 7
- Types . . . 7
- Best Practices . . . 7
- Extending . . . 7

Chat Coach Module **8**
- Features . . . 8
- Main Components . . . 8
  - `ChatInput` . . . 8
  - `ChatMessagesList` . . . 8
  - `ChatMessage` . . . 8
  - `QuickQuestions` . . . 8
  - `TypingIndicator` . . . 8
- Hooks . . . 8
  - `useChat` . . . 8
  - `useAutoScroll` . . . 8
- Store . . . 8
- Services . . . 9

Types . . . 9
Usage Example . . . 9
File Structure . . . 9
Notes . . . 9

Chat Coach Modal Module **9**
Features . . . 9
Main Components . . . 9
`AiChatModal` . . . 9
`ChatModalHeader` . . . 10
`ChatModalMessages` . . . 10
`ChatModalMessage` . . . 10
`ChatModalInput` . . . 10
`SuggestedQuestions` . . . 10
`TypingIndicatorModal` . . . 10
`ErrorMessage` . . . 10
Hooks . . . 10
`useChatModal` . . . 10
`useChatInput` . . . 10
Store . . . 10
Services . . . 10
Types . . . 10
Usage Example . . . 11
File Structure . . . 11
Notes . . . 11

Content Post Module **11**
Features . . . 11
Main Components . . . 11
`PostGrid` . . . 11
`PostCard` . . . 11
Hooks . . . 11
`useContentFeedback` . . . 11
Store . . . 12
Services . . . 12
Types . . . 12
Usage Example . . . 12
File Structure . . . 12
Notes . . . 12

Create Post Module **12**
Features . . . 12
Main Components . . . 13
`CreatePostModal` . . . 13
`DescriptionInput` . . . 13
`PlatformSelector` . . . 13
`ImageUpload` . . . 13
`TrendsSection` . . . 13
`ScheduleModal` . . . 13
`AISuggestion` . . . 13
`SuccessNotification` . . . 13
Hooks . . . 13
`usePost` . . . 13
Store . . . 13

Services . . . 13
Types . . . 14
Usage Example . . . 14
File Structure . . . 14
API Endpoints . . . 14
Notes . . . 14

Core Module **14**
Structure & Responsibilities . . . 14
Main Modules . . . 15
API . . . 15
Context . . . 15
Hooks . . . 15
Lib . . . 15
Store . . . 15
Usage Example . . . 15
Accessing the API Client . . . 15
Using the Theme Provider . . . 15
Using Redux Store . . . 16
Notes . . . 16

Shared Module **16**
Structure & Responsibilities . . . 16
Main Modules & Components . . . 16
Components . . . 16
Layout . . . 17
Pages . . . 17
Router . . . 17
Types . . . 17
Usage Example . . . 17
Importing Shared Components . . . 17
Using Layouts . . . 17
Using Table System . . . 17
Notes . . . 17

Marketin Agent

Project Structure

This project follows a modular architecture, organized primarily by feature (modules) with a shared layer for reusable elements. The structure is designed for scalability, maintainability, and effective team collaboration.

```
src/
  assets/             # Static resources (images, icons, etc.)
  core/               # Core infrastructure and shared services
  modules/            # Feature-based organization
  shared/             # Reusable components, hooks, and utilities
  App.tsx             # Main application component
  main.tsx            # Application entry point
  index.css           # Global styles
```

Modules Overview

modules/

Contains code organized by business feature. Each feature is self-contained and includes all aspects needed to implement its specific functionality.

```
modules/
  auth/                  # Authentication functionality
   components/          # Auth-specific components
   page/                # Auth-related pages
   hooks/               # Auth-specific hooks
   store/               # Redux state for auth
   types/               # TypeScript types for auth
   services/            # API services for auth
 ...
  campaigns/             # Campaign management
  chat-coach/            # AI marketing coach chat
  chat-coach-modal/      # Modal chat with AI coach
  content-post/          # Content feedback and posts
  create-post/           # Post creation and AI suggestions
  dashboard/             # Dashboard and analytics
```

Purpose: Each feature encapsulates all logic, components, pages, and state related to a specific business functionality. This enables parallel development and improves code cohesion.

core/

Contains core infrastructure and services shared across the application.

```
core/
  api/                   # API configuration and utilities
   apiClient.ts         # Configured HTTP client
   apiErrors.ts         # API error handling
  context/               # Shared React contexts (e.g., theme)
  hooks/                 # Core hooks (e.g., mobile detection, modals)
  lib/                   # Core utilities
  store/                 # Central Redux configuration
   store.ts             # Store setup
   rootReducer.ts       # Root reducer
   middleware.ts        # Custom middleware
```

Purpose: Provides the technical foundation and shared services for the entire app. Centralizes critical configurations and ensures consistency.

shared/

Contains reusable UI components, hooks, utilities, and types shared across modules.

```
shared/
  components/            # Reusable UI components
   button/              # Button variants
   cards/               # Card components
   dropdown/            # Dropdowns and menus
   error-state/         # Error display components
   header/              # App and site headers
   loading-state/       # Loading indicators
   table/               # Table and data display
```

```
    tooltip/             # Tooltip components
    ui/                  # Atomic UI elements (inputs, badges, etc.)
   layout/                # Layout and routing helpers
   pages/                 # Shared pages (e.g., Not Found)
   types/                 # Shared TypeScript types
    types.ts             # Common types
   router.ts              # Shared router utilities
```

Purpose: Provides a library of reusable UI components, hooks, and utilities to maintain visual and behavioral consistency throughout the app. Promotes code reuse and a coherent design system.

assets/

Contains static resources used throughout the application.

```
 assets/
   logo-mark.webp         # Logos and icons
   react.svg              # Framework icons
  ...
```

Purpose: Centralizes all static resources for easier management and optimization.

Development Guidelines

Code Organization

1. **Single Responsibility Principle**: Each file should have a single responsibility and reason to change.
2. **Modularity**: Organize code into cohesive modules with clear interfaces.
3. **Encapsulation**: Hide internal implementations, exporting only what's necessary.
4. **Reuse**: Extract common components, hooks, and utilities into the `shared` folder.

State Management

1. **Redux for global state**: Use Redux for state shared across multiple modules.
2. **React Context for UI state**: Use Context API for UI state shared within a component tree.
3. **useState/useReducer for local state**: For component-specific state.

Naming Conventions

1. **Components**: PascalCase (e.g., `UserProfile.tsx`)
2. **Hooks**: camelCase with "use" prefix (e.g., `useAuth.ts`)
3. **Utilities and services**: camelCase (e.g., `formatDate.ts`, `authService.ts`)
4. **Types/Interfaces**: PascalCase (e.g., `UserData.ts`)
5. **Redux files**: camelCase with descriptive suffix (e.g., `userSlice.ts`)

Redux Workflow

1. **Slices**: Define slices with reducers, actions, and selectors.
2. **Thunks**: Use thunks for async logic.
3. **Selectors**: Use selectors to access state.
4. **Custom hooks**: Encapsulate Redux logic in reusable hooks.

Best Practices

1. **Type Safety**: Use TypeScript for all code.
2. **Testing**: Write unit and integration tests for components and logic.
3. **Lazy Loading**: Implement lazy loading for large routes/modules.

4. **Documentation**: Document components, hooks, and functions with JSDoc.
5. **Performance**: Optimize rendering with React.memo, useMemo, and useCallback as needed.
6. **Consistent UI**: Use Tailwind CSS consistently, creating abstractions when necessary.
7. **Atomic Components**: Design components following atomic design principles (atoms, molecules, organisms).

Auth Module

This module handles authentication for the application, including login, registration, authentication state management, and route protection. It is designed to be modular, type-safe, and easy to integrate with the rest of the app.

Structure

```
src/modules/auth/
  components/
     RequireAuth.tsx         # Route guard for protected routes
  hooks/
     useAuth.ts              # Custom hook to access auth context
  page/
     authPage.tsx            # Login page UI and logic
  services/
     authService.ts          # API calls and service logic for auth
  store/
     authProvider.tsx        # React Context provider for auth state
     authSlice.ts            # Redux slice for authentication (optional/advanced)
  types/
      auth.types.ts           # TypeScript types for auth
```

Features

- **Login & Registration**: Handles user login and registration via API.
- **Auth State Management**: Stores authentication state (token, user info) using React Context and optionally Redux.
- **Route Protection**: `RequireAuth` component restricts access to authenticated users.
- **Type Safety**: All logic and data structures are strongly typed with TypeScript.
- **Reusable Hooks**: `useAuth` provides easy access to authentication state and actions.

Usage

1. AuthProvider

Wrap your app (or the part that needs authentication) with the `AuthProvider` to provide authentication context:

```
import { AuthProvider } from './modules/auth/store/authProvider';

function App() {
  return (
    <AuthProvider>
      {/* ...your routes/components */}
    </AuthProvider>
```

```
  );
}
```

2. Protecting Routes

Use the `RequireAuth` component to protect routes that require authentication:

```
import RequireAuth from './modules/auth/components/RequireAuth';

<Route
  path="/dashboard"
  element={
    <RequireAuth>
      <DashboardPage />
    </RequireAuth>
  }
/>
```

3. Accessing Auth State and Actions

Use the `useAuth` hook to access authentication state and actions (login, logout, etc.):

```
import { useAuth } from './modules/auth/hooks/useAuth';

const { isAuthenticated, login, logout, token } = useAuth();
```

4. Login Example

The `authPage.tsx` provides a ready-to-use login form. It uses the `login` function from the context and handles navigation after successful login.

5. API Service

All authentication API calls are handled in `authService.ts`. You can extend this service for registration, password reset, etc.

Types

All user and auth-related types are defined in `types/auth.types.ts` for consistency and type safety.

Best Practices

- Use the Context-based approach (`authProvider.tsx` and `useAuth.ts`) for simple apps.
- For advanced scenarios (e.g., global user state, async flows), use the Redux slice (`authSlice.ts`).
- Always use the provided hooks and context to access or mutate authentication state.

Extending

- Add registration, password reset, or social login by extending `authService.ts` and updating the context/slice as needed.
- Add more user fields or roles by updating the types in `auth.types.ts`.

Chat Coach Module

This module provides an AI-powered chat interface for marketing advice and support. It allows users to interact with "Mark AI", a digital marketing coach, to get instant answers, suggestions, and strategies for social media and content marketing.

Features

- Conversational chat interface with Mark AI
- Send and receive messages in real time
- Frequently asked questions (quick questions)
- Typing indicator for assistant responses
- Auto-scroll to latest message
- Error handling and loading states
- Integration with post creation modal

Main Components

`ChatInput`

A textarea input for composing and sending messages. Supports pressing Enter to send and Shift+Enter for new lines.

`ChatMessagesList`

Displays the list of chat messages, including user and assistant messages, and shows a typing indicator when the assistant is responding.

`ChatMessage`

Renders a single chat message, differentiating between user and assistant roles, and supports markdown rendering for assistant responses.

`QuickQuestions`

Shows a set of frequently asked questions that users can click to quickly send to Mark AI.

`TypingIndicator`

Displays an animated indicator when Mark AI is generating a response.

Hooks

`useChat`

Custom hook that manages chat state, sending messages, error handling, and chat history. It provides: - `messages`: Array of chat messages - `isLoading`: Loading state for assistant response - `error`: Error message (if any) - `sendMessage`: Function to send a message - `clearChatError`: Function to clear errors - `resetChatHistory`: Function to reset the chat

`useAutoScroll`

Custom hook to automatically scroll to the latest message in the chat.

Store

Redux slice (`chatSlice.ts`) manages the chat state, including async thunk for: - Sending messages to the backend AI service - Handling loading, error, and message states

Services

- `chatService.ts` provides the API call to send messages to the Mark AI backend endpoint.

Types

- All types and interfaces for chat messages, requests, and responses are defined in `types/chatTypes.ts`.

Usage Example

Import and use the `MarketingCoachChatPage` for a ready-to-use chat page:

```
import MarketingCoachChatPage from './page/MarketingCoachChatPage';

// ...
<MarketingCoachChatPage />
```

This page includes: - The chat interface - Quick questions - Post creation modal integration

File Structure

- `components/` UI components for the chat interface
- `hooks/` Custom React hooks for chat logic
- `services/` API service functions
- `store/` Redux slice and async thunks
- `types/` TypeScript types and interfaces
- `page/` Page-level component for the chat coach

Notes

- The module uses Redux Toolkit for state management.
- All UI components use Tailwind CSS for styling.
- The chat backend endpoint must be available and configured for full functionality.

Chat Coach Modal Module

This module provides a modal-based AI chat assistant ("Mark") for marketing support. It allows users to interact with Mark in a modal dialog, ask questions, and receive instant AI-powered marketing advice, content ideas, and campaign analysis.

Features

- Modal chat interface with Mark, the AI marketing assistant
- Send and receive messages in real time
- Suggested questions for quick interaction
- Typing indicator for assistant responses
- Error handling and loading states
- Auto-scroll to latest message

Main Components

`AiChatModal`

The main modal dialog component that wraps the chat experience, including header, messages, input, and suggested questions.

`ChatModalHeader`

Displays the assistant's avatar, name, and online status at the top of the modal.

`ChatModalMessages`

Shows the list of chat messages and the typing indicator. Handles auto-scrolling to the latest message.

`ChatModalMessage`

Renders a single chat message, differentiating between user and AI roles, and displays the timestamp.

`ChatModalInput`

Input field and send button for composing and sending messages to Mark.

`SuggestedQuestions`

Displays a set of suggested questions that users can click to quickly fill the input.

`TypingIndicatorModal`

Animated indicator shown when Mark is generating a response.

`ErrorMessage`

Displays error messages with an optional dismiss button.

Hooks

`useChatModal`

Custom hook that manages chat state, sending messages, error handling, and chat reset. It provides: - `messages`: Array of chat messages - `isTyping`: Loading state for assistant response - `error`: Error message (if any) - `sendMessage`: Function to send a message - `clearError`: Function to clear errors - `resetChat`: Function to reset the chat

`useChatInput`

Custom hook for managing the chat input value, sending messages, handling key presses, and setting suggested questions.

Store

Redux slice (`chatModalSlice.ts`) manages the chat state, including async thunk for: - Sending messages to the backend AI service - Handling loading, error, and message states

Services

- `chatModalService.ts` provides the API call to send messages to the Mark AI backend endpoint.

Types

- All types and interfaces for chat messages, requests, and responses are defined in `types/chatModalTypes.ts`.

Usage Example

Import and use the `AiChatModal` in your page or component:

```
import { AiChatModal } from './page/AiChatModal';

// ...
<AiChatModal isOpen={isModalOpen} onClose={() => setIsModalOpen(false)} />
```

This modal can be opened from any part of your app to provide instant marketing support.

File Structure

- `components/` UI components for the modal chat interface
- `hooks/` Custom React hooks for chat modal logic
- `services/` API service functions
- `store/` Redux slice and async thunks
- `types/` TypeScript types and interfaces
- `page/` Modal dialog component for the chat coach

Notes

- The module uses Redux Toolkit for state management.
- All UI components use Tailwind CSS for styling.
- The chat backend endpoint must be available and configured for full functionality.

Content Post Module

This module is responsible for displaying, managing, and providing feedback on social media posts. It is designed to be used within a marketing dashboard or content management platform, allowing users to view posts, see feedback, and interact with post-related features.

Features

- Fetch and display a grid of social media posts
- Show post details, author, date, likes, and feedback
- Error and loading state handling
- Modal integration for creating new posts and asking for AI feedback

Main Components

`PostGrid`

Displays a responsive grid of posts using the `PostCard` component for each post.

`PostCard`

Shows the details of a single post, including: - Author and avatar - Date - Number of likes - Post text - Feedback/AI comment - Link to the original post

Hooks

`useContentFeedback`

Custom hook that manages fetching posts, loading and error states, and clearing errors. It provides: - `posts`: Array of post objects - `loading`: Loading state - `error`: Error message (if any) - `clearError`: Function to clear the error

Store

Redux slice (`postSlice.ts`) manages the state for posts, including async thunk for: - Fetching posts from the backend or a local JSON file - Handling loading and error states

Services

- `postService.ts` provides the API call to fetch posts from `/posts.json` (can be replaced with a real backend endpoint).

Types

- All types and interfaces for posts and API responses are defined in `types/postTypes.ts`.

Usage Example

Import and use the `ContentFeedbackPage` for a ready-to-use page:

```
import ContentFeedbackPage from './page/ContentFeedbackPage';

// ...
<ContentFeedbackPage />
```

This page includes: - The post grid - Loading and error states - Modals for creating posts and asking for AI feedback

File Structure

- `components/` UI components for displaying posts
- `hooks/` Custom React hooks for post feedback logic
- `services/` API service functions
- `store/` Redux slice and async thunks
- `types/` TypeScript types and interfaces
- `page/` Page-level component for content feedback

Notes

- The module uses Redux Toolkit for state management.
- All UI components use Tailwind CSS for styling.
- The posts are loaded from a local JSON file by default, but the service can be adapted for a real backend.

Create Post Module

This module provides a complete UI and logic for creating, scheduling, and managing social media posts with AI-powered suggestions and image generation. It is designed for use in a marketing dashboard or social media management platform.

Features

- Select post type and target platforms
- AI-powered content suggestion for post descriptions
- AI-powered image generation for posts
- Upload custom images
- Trending hashtags suggestions for Instagram and LinkedIn
- Schedule posts for future publication

- Success notifications and loading states

Main Components

`CreatePostModal`

A modal dialog that contains the entire post creation form, including: - Post type selection - Platform selection - Description input (with AI suggestion) - Trends/hashtags section - Image upload and AI image generation - Schedule modal for post scheduling - Success notification

`DescriptionInput`

Textarea for post description with a button to get AI-generated suggestions.

`PlatformSelector`

Lets users select one or more social media platforms to target.

`ImageUpload`

Allows users to upload an image or generate one using AI based on the post description.

`TrendsSection`

Displays trending hashtags for Instagram and LinkedIn, allowing users to add them to the description.

`ScheduleModal`

A modal for selecting date and time to schedule the post.

`AISuggestion`

Displays a static or dynamic AI suggestion for optimal posting time and engagement tips.

`SuccessNotification`

Shows a notification when a post is created successfully.

Hooks

`usePost`

Custom hook that manages all state and actions for the post creation flow. It handles: - Form state (type, platforms, description, images, schedule) - Fetching trends - AI suggestions and image generation - Submitting and scheduling posts - Success and loading states

Store

Redux slice (`createPostSlice.ts`) manages the state for the post creation process, including async thunks for: - Creating a post - Getting AI suggestions - Generating images - Fetching trends

Services

- `createPostService.ts` provides API calls for creating posts, getting suggestions, generating images, and fetching trends from backend endpoints.

Types

- All types and constants (platforms, post types, state shape) are defined in `types/createPostTypes.ts`.

Usage Example

Import and use the `CreatePostModal` in your page:

```
import { CreatePostModal } from './components/CreatePostModal';

// ...
<CreatePostModal isOpen={isModalOpen} onClose={() => setIsModalOpen(false)} />
```

Or use the provided `CreatePostPage` for a ready-to-use page:

```
import { CreatePostPage } from './page/CreatePostPage';

// ...
<CreatePostPage />
```

File Structure

- `components/` UI components for the post creation flow
- `hooks/` Custom React hooks
- `services/` API service functions
- `store/` Redux slice and async thunks
- `types/` TypeScript types and constants
- `page/` Page-level component for post creation

API Endpoints

All API calls are routed through the backend defined in `createPostService.ts`. Endpoints include: - Create post - Get AI suggestion - Generate image - Get Instagram/LinkedIn trends

Notes

- The module uses Redux Toolkit for state management.
- All UI components use Tailwind CSS for styling.
- The AI features require backend endpoints to be available and configured.

Core Module

The `core` directory contains foundational utilities, API clients, context providers, hooks, and the global Redux store setup for the application. It provides essential building blocks and shared logic used throughout all feature modules.

Structure & Responsibilities

- **api/**: API client configuration, error handling, and abstraction for HTTP requests.
- **context/**: React context providers for global state, such as theme management.
- **hooks/**: Shared React hooks for device detection, modal management, and more.
- **lib/**: Utility functions and helpers (e.g., class name merging).
- **store/**: Redux store configuration, root reducer, and middleware.

Main Modules

API

- **apiClient.ts**: Configures an Axios instance with base URL, timeout, headers, authentication token injection, and standardized error handling. Exports a ready-to-use `apiClient` and an `ApiService` class for common HTTP operations (`get`, `post`, `put`, `patch`, `delete`).
- **apiErrors.ts**: Defines custom error classes (`ApiError`, `NetworkError`, `AuthenticationError`) and a set of standard error codes for consistent error handling across the app.

Context

- **theme-provider.tsx**: Provides a `ThemeProvider` React context for managing and persisting the application's color theme (light, dark, or system). Includes a `useTheme` hook for easy access to theme state and setter.

Hooks

- **useMobile.ts**: Custom hook to detect if the current device is mobile based on window width.
- **useModals.ts**: Custom hook to manage the open/close state of common modals (e.g., create post, ask Mark AI).

Lib

- **utils.ts**: Utility function `cn` for merging Tailwind CSS class names using `clsx` and `tailwind-merge`.

Store

- **middleware.ts**: Example Redux middleware for logging actions and state changes.
- **rootReducer.ts**: Combines all feature reducers (auth, posts, chat, create post, etc.) into a single root reducer for the Redux store.
- **store.ts**: Configures the Redux store with middleware and exports the store, `RootState`, and `AppDispatch` types.

Usage Example

Accessing the API Client

```python
import { apiService } from './api/apiClient';

// Example GET request
apiService.get('/endpoint').then(response => {
  // handle response
});
```

Using the Theme Provider

```
import { ThemeProvider } from './context/theme-provider';

<ThemeProvider>
  <App />
</ThemeProvider>
```

Using Redux Store

```
import { store } from './store/store';
import { Provider } from 'react-redux';

<Provider store={store}>
  <App />
</Provider>
```

Notes

- All code in `core` is intended to be generic and reusable across the application.
- The API client is pre-configured for authentication and error handling, but can be extended as needed.
- The Redux store is set up with logging middleware for development and debugging.
- Utility functions and hooks are designed to be composable and easy to use in any feature module.

Shared Module

The `shared` directory contains reusable UI components, layouts, types, and router exports that are used throughout the application. It provides a design system and common building blocks for consistent and efficient development across all feature modules.

Structure & Responsibilities

- **components/**: Reusable UI components (buttons, cards, tables, dropdowns, tooltips, etc.) and design system primitives.
- **layout/**: Layout components for pages and dashboards (e.g., sidebar, headers, outlet layouts).
- **pages/**: Shared pages such as the 404 Not Found page.
- **router.ts**: Centralized exports for shared components, layouts, and context providers for easy imports.
- **types/**: Shared TypeScript types and interfaces.

Main Modules & Components

Components

- **Sidebar**: Main sidebar navigation for the dashboard, with support for navigation groups, AI assistant, and user info.
- **AppHeader / SiteHeader**: Top navigation bars for the app and individual pages, supporting actions, search, and user menu.
- **Dropdowns**: Avatar dropdown, theme toggle, and other dropdown menus using Radix UI.
- **Cards**: KPI cards and other card-based UI elements.
- **Table**: Highly flexible table system with dynamic columns, filters, pagination, row actions, and view options.
- **Tooltip**: Hover tooltips for enhanced UX.
- **Button**: Standard and animated (shine-border) button components.
- **Form**: Form primitives and helpers for building forms with validation.
- **Dialog, Sheet, Popover, Accordion, Tabs, etc.**: Modal and interactive UI primitives based on Radix UI.
- **LoadingState / ErrorState**: Standardized loading and error display components.

Layout

- **dashboard-layout.tsx**: Main dashboard layout with sidebar and header.
- **page-outlet-layout.tsx**: Page layout with scrollable content and header actions.
- **breadcrumb-router.tsx**: Dynamic breadcrumb navigation based on the current route.

Pages

- **not-found-page.tsx**: 404 Not Found page for unmatched routes.

Router

- **router.ts**: Centralized exports for shared components, layouts, and context providers for easy and consistent imports across the app.

Types

- **types.ts**: Shared TypeScript types and interfaces (e.g., for header actions).

Usage Example

Importing Shared Components

```
import { AppHeader, Sidebar, KPICard, TooltipHover } from '@/shared/router';

<AppHeader />
<Sidebar />
<KPICard title="Engagement" value={1234} />
<TooltipHover title="Info">Hover me</TooltipHover>
```

Using Layouts

```
import DashboardLayout from '@/shared/layout/dashboard-layout';

<DashboardLayout>
  {/* page content here */}
</DashboardLayout>
```

Using Table System

```
import { DynamicTable, createColumns } from '@/shared/components/table/dynamic-table';

const columns = createColumns({ columns: [...] });
<DynamicTable columns={columns} data={data} />
```

Notes

- All components are styled with Tailwind CSS and use Radix UI for accessibility and composability.
- The design system is intended to be consistent and easily extendable.
- Shared types and router exports help maintain type safety and import simplicity across the codebase.